CPSC 375 High-Performance Computing

Lecture 8

Signals

Linux IPC

- Pipes
- Message Queues
- Shared memory
- Semaphores
- Signals
- Sockets

Signals

- A *signal* is a small message that notifies a process that an event of some type has occurred in the system
  - e.g., exceptions and interrupts
  - Sent from the kernel (sometimes at the request of another process) to a process
  - Signal type is identified by small integer ID's (1-30)
  - Only information in a signal is its ID and the fact that it arrived

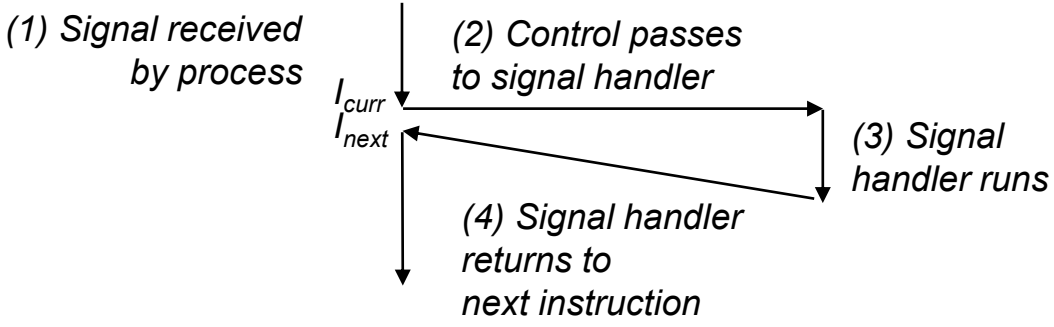
ID	*Name*	*Default Action*	*Corresponding Event*
2	SIGINT	Terminate	User typed ctrl-c
9	SIGKILL	Terminate	Kill program
11	SIGSEGV	Terminate & Dump	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

Sending a Signal

- Kernel *sends* a signal to a *destination process* by updating some state in the context of the destination process
- Kernel sends a signal for one of the following reasons:
  - Kernel has detected a system *event* such as
    - Divide-by-zero (SIGFPE) or
    - Termination of a child process (SIGCHLD)
  - Another process has invoked the `kill` system call to explicitly request the kernel to send a signal to the destination process

Receiving a Signal

- A destination process *receives* a signal when it is forced by the kernel to react in some way.
- Some possible ways to react:
  - *Ignore* the signal (do nothing)
  - *Terminate* the process
  - *Catch* the signal by executing a user-level function called *signal handler*



Pending and Blocked Signals

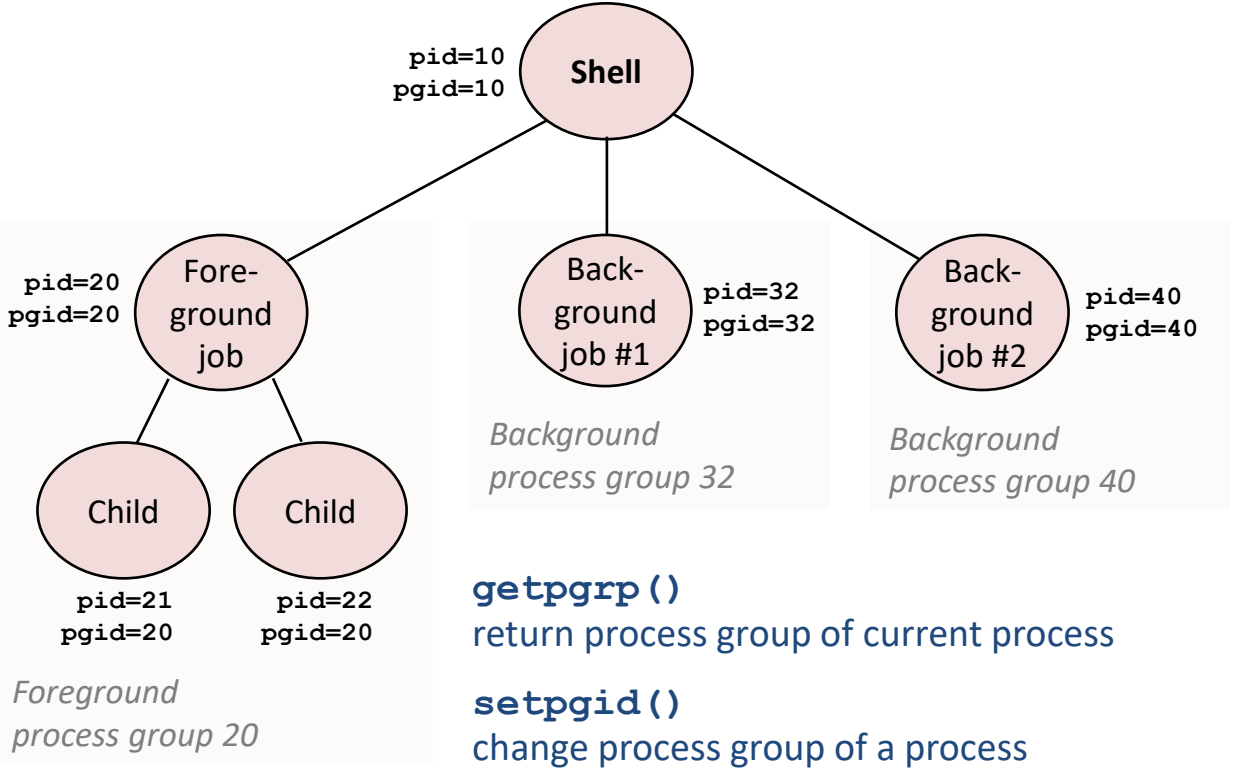
- A signal is *pending* if sent but not yet received.
  - There can be at most one pending signal of any particular type.
  - Important: Signals are not queued.
    - If a process has a pending signal of type *k*, then subsequent signals of type *k* that are sent to that process are discarded.
  - A pending signal is received at most once.
- A process can *block* the receipt of certain signals.
  - Blocked signals can be delivered, but will not be received until the signal is unblocked.

Pending/Blocked Bits

- Kernel maintains bit vectors in the context of each process
  - `pending`: represents the set of pending signals
    - Kernel sets bit *k* in `pending` when a signal of type *k* is delivered
    - Kernel clears bit *k* in `pending` when a signal of type *k* is received
  - `blocked`: represents the set of blocked signals
    - Can be set and cleared by using the `sigprocmask` function
    - Also referred to as the *signal mask*.

Sending Signals: Process Groups

- Every process belongs to exactly one process group



`getpgrp()`
return process group of current process

`setpgid()`
change process group of a process

Sending Signals with `kill` Program

- `kill` program sends arbitrary signal to a process or process group
- Examples

```
$ kill -9 24818
```

Send SIGKILL to process 24818

```
$ kill -9 -24817
```

Send SIGKILL to every process in process group 24817

Sending Signals from the Keyboard

- Ctrl-C
  - Sends SIGINT to every job in the foreground process group
  - Default action is to terminate each process
- Ctrl-Z
  - Causes the kernel to send SIGTSTP to every job in the foreground process group.
  - Default action is to stop (suspend) each process

Example of `ctrl-c` and `ctrl-z`

```
$ ./prog
Child: pid=28108 pgrp=28107
Parent: pid=28107 pgrp=28107
<types ctrl-z>
Suspended
$ ps w
  PID TTY      STAT   TIME COMMAND
27699 pts/8    Ss     0:00 -tcsh
28107 pts/8    T      0:01 ./prog
28108 pts/8    T      0:01 ./prog
28109 pts/8    R+     0:00 ps w

$ fg
./prog
<types ctrl-c>
$ ps w
  PID TTY      STAT   TIME COMMAND
27699 pts/8    Ss     0:00 -tcsh
28110 pts/8    R+     0:00 ps w
```

STAT (process state) Legend:

First letter:
S: sleeping
T: stopped
R: running

Second letter:
s: session leader
+: foreground proc group

See "`man ps`" for details.

Sending Signals with `kill` Function

```
void func_kill()
{
    pid_t pid[N];
    int i;
    int child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            /* Child: Infinite Loop */
            while(1)
                ;
        }

    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                   wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}
```

Default Actions

- Each signal type has a predefined *default action*, which is one of:
  - The process terminates
  - The process terminates and dumps core
  - The process stops until restarted by a SIGCONT signal
  - The process ignores the signal

Installing Signal Handlers

- The `signal` function modifies the default action associated with the receipt of signal `signum`:

```
handler_t *signal(int signum, handler_t *handler)
```

- Different values for `handler`:
  - SIG_IGN: ignore signals of type `signum`
  - SIG_DFL: revert to the default action on receipt of signals of type `signum`
  - Or …

Installing Signal Handlers, cont'd

```
handler_t *signal(int signum, handler_t *handler)
```

Otherwise, `handler` is the address of a user-level *signal handler*

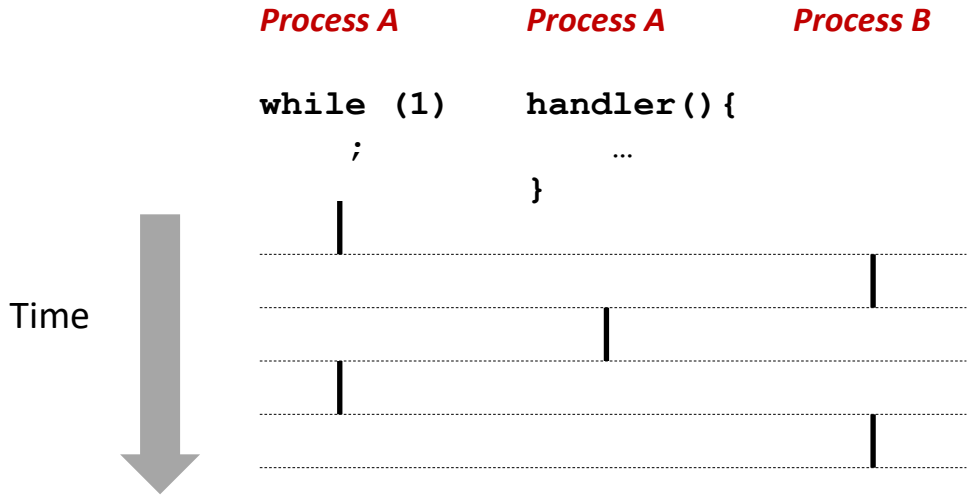
- Called when process receives signal of type `signum`
- Referred to as *"installing"* the handler
- Executing handler is called *"catching"* or *"handling"* the signal
- When the handler executes its return statement, control passes back to instruction in the control flow of the process that was interrupted by receipt of the signal.

Signal Handling Example

```
void handler(int sig) /* SIGINT handler */
{
    printf("Caught SIGINT\n");
    exit(0);

int main()
{
    /* Install the SIGINT handler */
    if (signal(SIGINT, handler) == SIG_ERR)
        unix_error("signal error");

    pause(); /* Wait for the receipt of a signal */

    exit(0);
}
```

Signals Handlers as Concurrent Flows

- A signal handler is a separate logical flow (not process) that runs concurrently with the main program



Nested Signal Handlers

- Handlers can be interrupted by other handlers